

AAP

AI Administration Platform

Document de conception — vision, produit, architecture, sécurité et technologies

Positionnement : plateforme SaaS d'administration intelligente et dynamique pour applications. AAP n'est pas un chatbot classique.

Objectif personnel : approfondir Python backend, Django, PostgreSQL, sécurité, LLM, RAG, tool calling, MCP, Docker, Redis/Celery et CI/CD à travers un projet réel.

1. Objectif du projet

Créer une plateforme SaaS permettant aux entreprises et développeurs d'intégrer rapidement une interface d'administration intelligente à leurs applications, sans développer manuellement une multitude de pages, tableaux, filtres, recherches et dashboards.

Un utilisateur peut interroger les données de son application en langage naturel. AAP comprend la demande, identifie les données nécessaires, génère une requête SQL, applique plusieurs couches de sécurité, l'exécute en lecture seule sur la base du client et présente le résultat sous forme de tableau, graphique ou résumé.

2. Problème

Les interfaces d'administration sont souvent longues à développer, répétitives, spécifiques à chaque application et coûteuses à maintenir.

Pour chaque besoin, le développeur doit souvent créer frontend, endpoints backend, requêtes SQL, filtres, permissions, tests et UI.

Les utilisateurs métier ne connaissent généralement pas SQL et doivent dépendre des développeurs pour rechercher une information ou obtenir un nouveau filtre.

Les applications évoluent : une nouvelle table ou relation peut nécessiter une nouvelle page

d'administration avec l'approche traditionnelle.

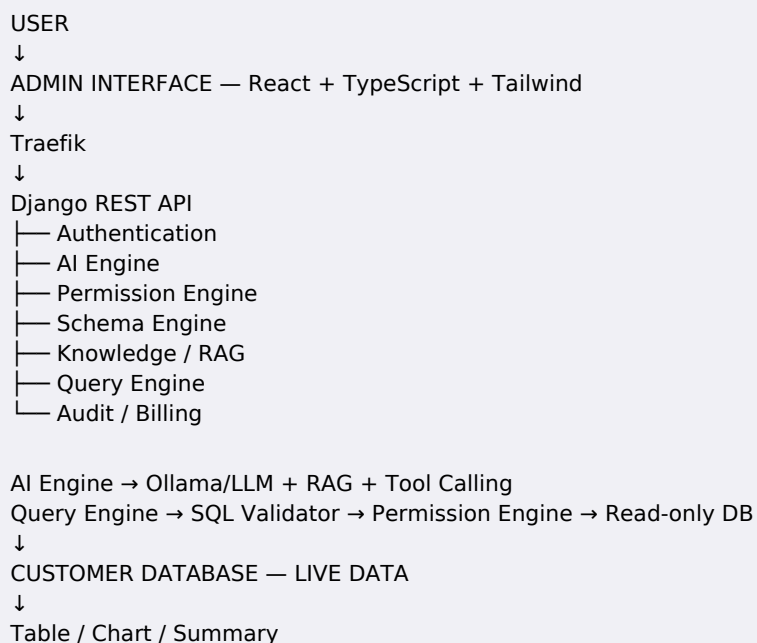
3. Solution

AAP devient une couche intelligente entre l'application du client et ses utilisateurs administrateurs.

Flux : question utilisateur → compréhension LLM → identification des données → génération SQL → validation sécurité → permissions → base client → résultat live → table/graphique/résumé.

Exemple : « Montre-moi les 20 clients ayant dépensé le plus ce mois-ci. » AAP sélectionne les tables pertinentes, génère et valide le SQL, vérifie les permissions, interroge la base du client et restitue le résultat.

4. Architecture générale



5. Utilisation de la base client — 3 méthodes

5.1 Connexion directe — MVP

Le client fournit une connexion PostgreSQL dédiée : host, port, database, username et password. AAP découvre le schéma puis exécute uniquement des requêtes autorisées. Le compte doit être read-only, idéalement avec SELECT seulement.

Avantages : simple, rapide et données toujours à jour. Inconvénients : connexion externe à sécuriser et configuration réseau à prévoir.

5.2 AAP Connector

Le client installe un petit service AAP dans son infrastructure, par exemple dans un container Docker. Les credentials restent côté client. Le connector expose des opérations contrôlées comme `get_schema()` et `execute_readonly_query()`.

5.3 API / MCP

Le client peut refuser l'accès direct à sa DB et exposer une API métier. AAP utilise alors des tools tels que `get_users()`, `get_orders()` ou `get_sales()`. MCP pourra ensuite standardiser les intégrations avec bases, logs, monitoring, GitHub et APIs externes.

Ordre recommandé : V1 PostgreSQL direct → V2 Connector → V3 API/MCP.

6. Source de vérité et données dynamiques

AAP ne doit pas copier en permanence users, orders, payments, products ou autres données métier. La base du client reste la source de vérité. AAP stocke principalement la structure, les permissions, la configuration, les connaissances et les historiques nécessaires.

Customer DB = Source of Truth
AAP = couche intelligente + sécurité + orchestration
Question → requête live → Customer DB → résultat actuel

Des résultats peuvent être conservés temporairement pour affichage ou cache, avec des contrôles de sécurité, mais ils ne doivent pas devenir une copie permanente des données du client.

7. Tables principales de la base AAP

Domaine	Tables
Identity	users, roles, permissions, user_roles
Organizations	organizations, organization_members
Applications	applications
Connections	database_connections, api_connections
Schema	database_schemas, database_tables, database_columns, database_relationships
Permissions	data_permissions, table_permissions, column_permissions
AI	ai_assistants, ai_conversations, ai_messages, ai_queries, ai_tool_calls
Knowledge	knowledge_sources, documents, document_chunks, embeddings
Audit	audit_logs, security_events, query_logs
Billing	plans, subscriptions, usage_records

8. Ce qu'AAP peut stocker

Métadonnées : tables, colonnes, types, relations, descriptions et informations de schéma.

Configuration : applications, connexions, permissions, rôles et configuration AI.

AI : conversations, messages, historique des requêtes et tool calls.

RAG : documents, chunks, embeddings, documentation, règles métier et descriptions enrichies du schéma.

Sécurité : audit logs, security events et informations d'authentification nécessaires.

Billing : plans, subscriptions et usage.

À ne pas stocker comme copie permanente : users, orders, payments, products et autres données métier du client.

9. Modules fonctionnels

Identity & Organization

- Users, organizations, teams, roles et permissions.

Application Management

- Connexion de plusieurs applications à une organisation.

Data Connections

- PostgreSQL, API, Connector et plus tard MCP.

Schema Management

- Découverte des tables, colonnes, types et relations.

AI Query Engine

- Natural language → compréhension → contexte → génération SQL → validation → exécution.

Permission Engine

- Contrôle de l'utilisateur, de l'application, des tables et des colonnes accessibles.

Knowledge / RAG

- Documentation, règles métier, descriptions de schéma et contexte applicatif.

Analytics

- Tables, graphiques, KPIs, résumés et requêtes sauvegardées.

Audit & Security

- Historique des actions, requêtes, tables consultées et événements de sécurité.

AI Operations — futur

- Analyse des logs, erreurs, métriques et traces afin d'aider au diagnostic.

10. Moteur AI

```
USER QUESTION
↓
Intent Analyzer
↓
Context Retrieval — Schema + RAG
↓
LLM
↓
Tool Calling / SQL Generation
↓
SQL Validator
↓
```

```
Permission Engine
↓
Query Executor
↓
Customer Database
```

Principe : le LLM propose ; AAP décide ; la base de données exécute. Le LLM ne doit jamais être considéré comme une couche de sécurité.

11. Sécurité

- Le LLM reçoit l'instruction de produire uniquement des opérations de lecture.
- Un validator SQL rejette INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE et autres opérations interdites.
- Les permissions AAP contrôlent les tables et colonnes accessibles.
- Le compte PostgreSQL utilisé par AAP doit lui-même être read-only.
- Les requêtes et accès doivent être audités.
- Les credentials doivent être protégés/chiffrés et ne jamais apparaître dans les logs.

12. Technologies et pourquoi

Technologie	Pourquoi
React	Interface d'administration dynamique.
Vite	Développement et build frontend rapides.
TypeScript	Typage et maintenabilité.
Tailwind CSS	Design system rapide et cohérent.
Django	Backend Python mature, ORM, auth, sécurité et architecture.
Django REST Framework	API REST pour frontend et intégrations.
PostgreSQL	Base relationnelle robuste du cœur de la plateforme.
pgvector	Recherche et stockage vectoriel pour le RAG.
Redis	Cache, rate limiting et broker.
Celery	Tâches longues, synchronisation de schéma et jobs périodiques.
Ollama	Exécution locale des LLM.
Qwen / Llama	Modèles locaux à expérimenter et comparer.
RAG	Contexte documentaire et métier pour le LLM.
Tool Calling	Utilisation contrôlée de fonctions/outils.
MCP	Standardisation des intégrations plus tard.
Docker	Reproductibilité, isolation et déploiement.
Traefik	Reverse proxy et routing.
GitHub Actions	Tests,lint,build, sécuritéetCI/CD.

13. Architecture backend recommandée

Commencer par un modular monolith Django, plutôt que par de nombreux microservices. Cela garde le système maîtrisable tout en permettant d'approfondir le backend Python.

```
backend/
├── apps/
│   ├── accounts/
│   ├── organizations/
│   ├── applications/
│   ├── connections/
│   └── schemas/
```

```
| | permissions/
| | ai/
| | knowledge/
| | queries/
| | audit/
| | billing/
| | config/
| | manage.py
```

Des composants pourront devenir des services indépendants uniquement lorsqu'une vraie raison technique ou opérationnelle apparaît.

14. Business model

AAP peut être commercialisé comme un SaaS B2B.

Plan	Positionnement
Free / Developer	1 application, requêtes/utilisateurs limités, fonctionnalités de base.
Pro	Plusieurs applications, davantage de requêtes, RAG, analytics et permissions avancées.
Business	Multi-applications, audit avancé, connectors, permissions avancées et monitoring.
Enterprise	SSO, on-premise/private deployment, MCP, intégrations personnalisées et SLA.

15. Clients cibles

- Startups voulant éviter de construire une administration complète dès le début.
- SaaS companies avec beaucoup de données opérationnelles.
- Agences web développant plusieurs applications.
- Équipes backend voulant réduire le travail répétitif lié aux interfaces admin.
- Entreprises et équipes internes : support, operations, sales, management, finance et analytics.

16. Ce que le client achète

Le client n'achète pas simplement un chatbot. Il achète du temps de développement économisé, une interface d'administration intelligente, un accès contrôlé aux données de son application et une exploration plus rapide pour les équipes métier.

17. Roadmap

Phase 1 — Core — Authentication, Organizations, Applications, connexion PostgreSQL et schema discovery.

Phase 2 — AI Query — Natural language, LLM, génération SQL, validation SQL et exécution read-only.

Phase 3 — Admin UI — Tables, charts, KPIs, saved queries et query history.

Phase 4 — Security — Roles, table permissions, column permissions, audit logs et rate limiting.

Phase 5 — RAG — Documentation, embeddings, pgvector et règles métier.

Phase 6 — Integrations — Connector, API et MCP.

Phase 7 — Operations — Logs, métriques, erreurs et investigation assistée par AI.

18. Nom du projet

Nom retenu : AAP — AI Administration Platform.

Ce nom est plus cohérent que « AIA — Intelligent Assistant » car le produit n'est plus principalement un assistant conversationnel. Son objectif est l'administration intelligente, l'accès contrôlé aux données,

l'analyse et, à terme, les opérations applicatives.

19. Résumé exécutif

AAP est une plateforme SaaS permettant aux entreprises d'ajouter une administration intelligente à leurs applications. Elle se connecte aux données du client, comprend son schéma, applique des permissions, utilise un LLM pour interpréter les demandes en langage naturel et génère des requêtes en lecture seule. Les requêtes sont exécutées directement sur la source de données du client afin de conserver des résultats dynamiques et à jour.

AAP ne remplace pas la base de données du client et ne doit pas devenir une copie de ses données métier. Il stocke principalement métadonnées, configurations, permissions, connaissances, embeddings, historiques et journaux nécessaires au fonctionnement et à l'audit.

Stack cible : React + TypeScript + Vite + Tailwind → Traefik → Django/DRF → PostgreSQL + pgvector + Redis + Celery → Ollama/LLM + RAG + Tool Calling → Customer DB read-only.

Vision long terme : faire évoluer l'AI Admin vers une plateforme d'AI Operations capable d'analyser logs, métriques, erreurs et intégrations externes via Connector, API et MCP.

Phrase de présentation : « Nous développons une plateforme SaaS permettant aux entreprises d'intégrer une administration intelligente à leurs applications. Au lieu de développer manuellement des interfaces, filtres et requêtes pour chaque besoin métier, les utilisateurs peuvent interroger leurs données en langage naturel. La plateforme analyse le schéma autorisé de l'application, génère et valide des requêtes en lecture seule, puis récupère les données directement depuis la source du client afin de fournir des résultats dynamiques sous forme de tableaux, graphiques et analyses. »

Projet d'apprentissage : Django/PostgreSQL, sécurité, permissions, Celery/Redis, SQL dynamique, vector search/RAG, tool calling, LLM locaux avec Ollama, MCP, Docker, CI/CD et architecture de production.